

Investigación y Análisis: Principios SOLID

Nombre: Manuel Jose Reyes Ubiera

Fecha: 3/6/2026

1. ¿Qué significa SOLID?

SOLID es un conjunto de principios de diseño de programación orientada a objetos que ayudan a crear código más organizado, flexible y fácil de mantener.

S - Single Responsibility Principle (SRP)

Significa Principio de Responsabilidad Única.

Una clase debe tener una sola responsabilidad y una sola razón para cambiar.

O - Open/Closed Principle (OCP)

Significa Principio Abierto/Cerrado.

Las clases deben permitir agregar nuevas funcionalidades sin modificar el código existente.

L - Liskov Substitution Principle (LSP)

Significa Principio de Sustitución de Liskov.

Una clase hija debe poder reemplazar a su clase padre sin romper el funcionamiento del programa.

I - Interface Segregation Principle (ISP)

Significa Principio de Segregación de Interfaces.

Una clase no debe estar obligada a implementar métodos que no necesita.

D - Dependency Inversion Principle (DIP)

Significa Principio de Inversión de Dependencias.

Las clases deben depender de abstracciones y no de implementaciones concretas.

2. Importancia de SOLID

SOLID es importante porque permite crear sistemas más fáciles de modificar y ampliar.

Proyectos grandes

Ayuda a dividir el código en partes independientes, evitando que un cambio pequeño afecte todo el sistema.

Problemas sin SOLID

Cuando no se aplican estos principios aparecen problemas como:

- Código difícil de entender.
- Muchas dependencias.
- Cambios que dañan otras partes.
- Dificultad para realizar pruebas.

Influencia

Mantenimiento

Facilita corregir errores y agregar funciones.

Reutilización

Permite usar clases y componentes en diferentes partes.

Escalabilidad

Permite que el sistema crezca sin volverse complicado.

Pruebas

Las clases separadas son más fáciles de probar.

Trabajo en equipo

Los programadores pueden trabajar en diferentes módulos sin afectar todo el proyecto.

Conceptos Clave

Acoplamiento

Es el nivel de dependencia entre clases. Menor acoplamiento significa más flexibilidad.

Cohesión

Es qué tan relacionadas están las tareas dentro de una clase. Una clase con alta cohesión hace una tarea específica.

Refactorización

Es mejorar el código existente sin cambiar lo que hace.

Escalabilidad

Es la capacidad de un sistema para crecer.

Mantenibilidad

Es qué tan fácil es modificar y corregir un programa.

Abstracción

Es ocultar detalles innecesarios y mostrar solamente lo importante.

Dependencias

Son relaciones donde una clase necesita otra para funcionar.

3. Parte práctica: Refactorización guiada

Parte 1 - Single Responsibility Principle (SRP)

Código Refactorizado

```
public class GeneradorReporte
{
    public void Generar()
    {
        Console.WriteLine("Generando reporte...");
    }
}

public class GuardadorArchivo
{
    public void Guardar()
    {
        Console.WriteLine("Guardando archivo...");
    }
}

public class EnviadorCorreo
{
    public void Enviar()
    {
        Console.WriteLine("Enviando correo...");
    }
}
```

Análisis

El diseño original tenía varias responsabilidades dentro de una sola clase.

La refactorización permite separar tareas y hacer cambios sin afectar otras partes.

Si el sistema crece sería difícil mantener una clase con demasiadas funciones.

Parte 2 - Open Closed Principle (OCP)

Código Refactorizado

```
public interface IDescuento
{
    double Calcular(double monto);
}

public class ClienteRegular : IDescuento
{
    public double Calcular(double monto)
    {
        return monto * 0.05;
    }
}

public class ClienteVIP : IDescuento
{
    public double Calcular(double monto)
    {
        return monto * 0.10;
    }
}
```

Análisis

El código original necesitaba modificar la clase cada vez que aparecía un cliente nuevo.

Con polimorfismo podemos agregar nuevos descuentos creando nuevas clases sin cambiar el código existente.

Parte 3 - Liskov Substitution Principle (LSP)

Código Refactorizado

```
public interface IVuela
{
    void Volar();
}
```

```
public class Aguila : IVuela
{
    public void Volar()
    {
        Console.WriteLine("Volando...");
    }
}

public class Pinguino
{
    public void Nadar()
    {
        Console.WriteLine("Nadando...");
    }
}
```

Análisis

El pingüino violaba LSP porque heredaba un comportamiento que no podía cumplir.

Separando comportamientos evitamos errores y hacemos una mejor jerarquía de clases.

Parte 4 - Interface Segregation Principle (ISP)

Código Refactorizado

```
public interface ITrabajador
{
    void Trabajar();
}

public interface IComedor
{
    void Comer();
}

public class Robot : ITrabajador
{
    public void Trabajar()
    {
        Console.WriteLine("Trabajando...");
    }
}
```

Análisis

La interfaz anterior obligaba al robot a tener una función que no necesitaba.

Dividir interfaces permite que cada clase implemente solamente lo necesario.

Parte 5 - Dependency Inversion Principle (DIP)

Código Refactorizado

```
public interface IDatabase
{
    void Guardar();
}

public class MySQLDatabase : IDatabase
{
    public void Guardar()
    {
        Console.WriteLine("Guardando en MySQL");
    }
}

public class UsuarioService
{
    private IDatabase db;

    public UsuarioService(IDatabase database)
    {
        db = database;
    }

    public void CrearUsuario()
    {
        db.Guardar();
    }
}
```

Análisis

La clase depende de una interfaz y no de una base de datos específica.

Esto permite cambiar MySQL por otra tecnología fácilmente.

También facilita hacer pruebas porque se pueden usar implementaciones diferentes.

Reflexión Final

El principio más difícil fue LSP porque requiere diseñar correctamente las relaciones entre clases.

El principio que aporta más valor es SRP porque evita clases demasiado grandes.

Después de esta actividad entiendo que un código rápido puede funcionar, pero un código bien diseñado será más fácil de mantener y mejorar.

La diferencia principal es que un código bien diseñado piensa en futuros cambios.